



山东大学

信息科学与工程学院

2024—2025 学年第二学期

实 验 报 告

课程名称: 数据结构与算法导论

实验名称: 实验 7: 哈夫曼的树的实现

专 业 班 级 通信工程 三班

学 生 学 号 _____

学 生 姓 名 _____

实 验 时 间 2025 年 5 月 26 日

实验报告

【实验目的】

掌握Huffman树的概念、存储结构；掌握建立Huffman树和Huffman编码的方法；

【实验原理】

哈夫曼编码是一种基于二叉树的前缀编码方法，通过最优二叉树（带权路径最短）实现数据高效压缩，即为权重乘以路径长最小。因此，我们可以知道，哈夫曼的树是一个带权的二叉树，我们定义如下：（最后通过数组实现）

```
typedef struct {
    int weight;           // 权值
    int parent;           // 父节点索引
    int lchild, rchild;   // 左右子节点索引
} HTNode, *HuffmanTree;
```

该如何使带权路径值最小呢(即构建哈夫曼的树呢)，我们通过迭代实现，步骤如下：

- (1) 选择未选择且权值最小的两个节点构成一个子树
- (2) 根节点的值子树两个节点值之和形成一个新节点
- (3) 向上继续迭代求根节点

对生成的哈夫曼的树进行编码也是从叶子节点到根节点进行遍历，根据当前节点是左孩子还是右孩子进行逆序编码，步骤如下：

- (1) 从数组末尾开始编码开始循环求编码
- (2) 若是左孩子编'0'，右孩子编'1'.
- (3) 将父节点变成子节点重复以上操作，直到到达根节点

【实验内容与要求】

内容：

利用动态分配数组存储哈夫曼树,设计一组输入数据（假定为一组整数），能够对其进行如下

操作：

- 1) 创建一个新的顺序表，实现动态空间分配的初始化；
- 2) 对输入的数据构造成一棵 Huffman 树；
- 3) 根据生成的 Huffman 树进行 Huffman 编码；
- 4) 实现对输入数据的 Huffman 编码输出。

要求：

- 1) 实现 Huffman 树的生成；
- 2) 完成 Huffman 编码的输出。

【实验源程序】

源程序：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// 定义赫夫曼树节点结构
typedef struct {
    int weight;           // 权值
    int parent;           // 父节点索引
    int lchild, rchild;   // 左右子节点索引
} HTNode, *HuffmanTree;

// 定义赫夫曼编码结构
typedef char **HuffmanCode;

// 选择两个权值最小且未被选中的节点
void Select(HuffmanTree HT, int n, int *s1, int *s2) {
    int min1 = 0x7FFFFFFF, min2 = 0x7FFFFFFF;
    *s1 = *s2 = 0;

    for (int i = 1; i <= n; i++) {
        if (HT[i].parent == 0) {
            if (HT[i].weight < min1) {
                min2 = min1;
                *s2 = *s1;
                min1 = HT[i].weight;
                *s1 = i;
            } else if (HT[i].weight < min2) {
                min2 = HT[i].weight;
            }
        }
    }
}
```

```

        *s2 = i;
    }
}

// 创建赫夫曼树
void CreateHuffmanTree(HuffmanTree *HT, int *w, int n) {
    if (n <= 1) return;
    int m = 2 * n - 1; // 赫夫曼树的总节点数
    *HT = (HuffmanTree)malloc((m + 1) * sizeof(HTNode)); // 0号单元未用

    // 初始化前 n 个节点为叶子节点
    for (int i = 1; i <= n; i++) {
        (*HT)[i].weight = w[i-1];
        (*HT)[i].parent = 0;
        (*HT)[i].lchild = 0;
        (*HT)[i].rchild = 0;
    }

    // 初始化剩余节点
    for (int i = n + 1; i <= m; i++) {
        (*HT)[i].weight = 0;
        (*HT)[i].parent = 0;
        (*HT)[i].lchild = 0;
        (*HT)[i].rchild = 0;
    }

    // 构建赫夫曼树
    for (int i = n + 1; i <= m; i++) {
        int s1, s2;
        Select(*HT, i - 1, &s1, &s2);
        (*HT)[s1].parent = i;
        (*HT)[s2].parent = i;
        (*HT)[i].lchild = s1;
        (*HT)[i].rchild = s2;
        (*HT)[i].weight = (*HT)[s1].weight + (*HT)[s2].weight;
    }
}

// 创建赫夫曼编码
void CreateHuffmanCode(HuffmanTree HT, HuffmanCode *HC, int n) {
    *HC = (HuffmanCode)malloc((n + 1) * sizeof(char *));

```

```

char *cd = (char *)malloc(n * sizeof(char)); // 临时存储编码
cd[n - 1] = '\\0'; // 编码结束符

for (int i = 1; i <= n; i++) {
    int start = n - 1; // 编码起始位置
    int c = i;         // 当前节点
    int f = HT[i].parent; // 父节点

    // 从叶子到根逆向求编码
    while (f != 0) {
        --start;
        if (HT[f].lchild == c) cd[start] = '0';
        else cd[start] = '1';
        c = f;
        f = HT[f].parent;
    }

    // 为第 i 个字符分配编码空间并复制
    (*HC)[i] = (char *)malloc((n - start) * sizeof(char));
    strcpy((*HC)[i], &cd[start]);
}

free(cd); // 释放临时空间
}

int main() {
    int n;
    printf("请输入数据个数: ");
    scanf("%d", &n);

    int *w = (int *)malloc(n * sizeof(int));
    printf("请输入%d 个整数权值:\\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &w[i]);
    }

    HuffmanTree HT;
    CreateHuffmanTree(&HT, w, n);

    HuffmanCode HC;
    CreateHuffmanCode(HT, &HC, n);

    printf("\\n 赫夫曼编码结果:\\n");
}

```

```

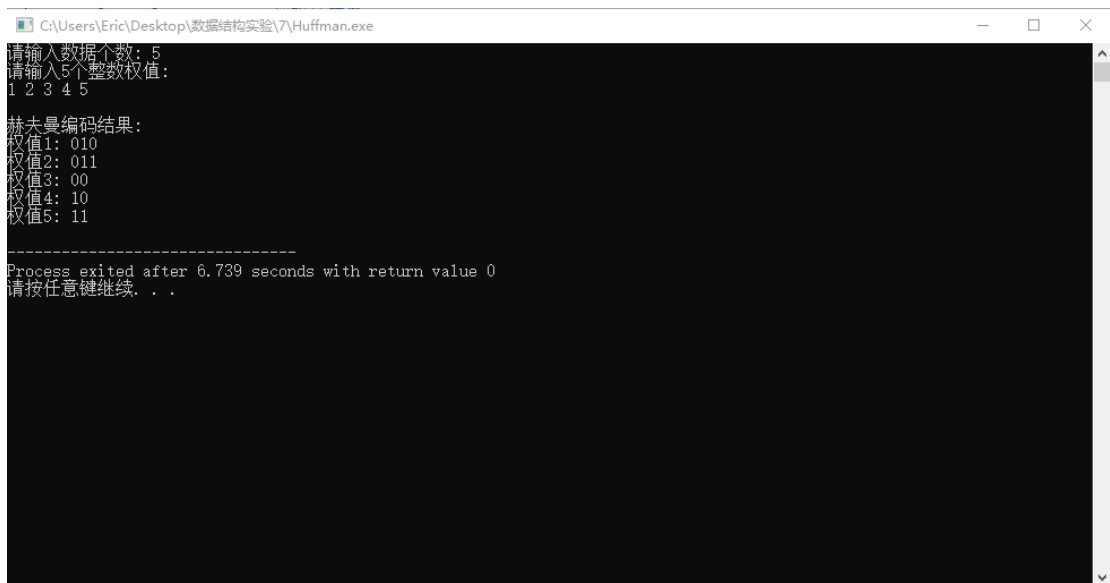
    for (int i = 1; i <= n; i++) {
        printf("权值%d: %s\n", HT[i].weight, HC[i]);
    }

    // 释放内存
    for (int i = 1; i <= n; i++) free(HC[i]);
    free(HC);
    free(HT);
    free(w);

    return 0;
}

```

【实验结果】



```

C:\Users\Eric\Desktop\数据结构实验\7\Huffman.exe
请输入数据个数: 5
请输入5个整数权值:
1 2 3 4 5

赫夫曼编码结果:
权值1: 010
权值2: 011
权值3: 00
权值4: 10
权值5: 11

-----
Process exited after 6.739 seconds with return value 0
请按任意键继续. . .

```

图 1 输出哈夫曼编码值

我们输入的权值分别是 1, 2, 3, 4, 5, 因此, 由哈夫曼编码画出哈夫曼的树的结构如下:

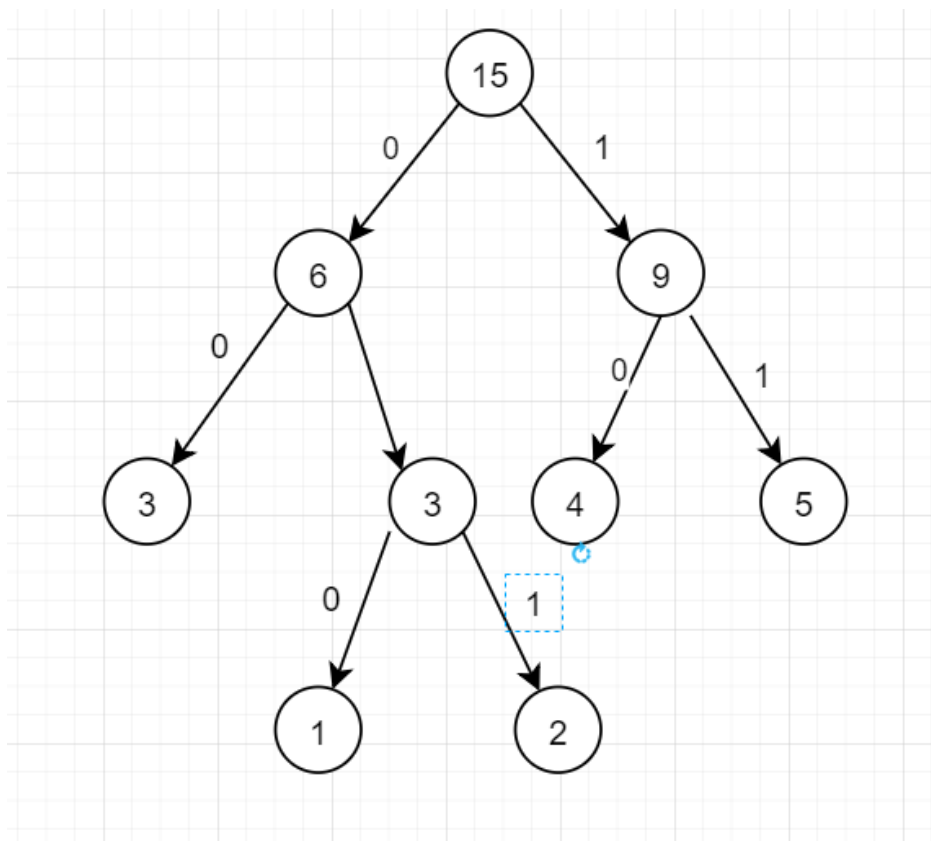


图 2 生成的哈夫曼的树

【实验心得】

本次实验，我们成功实现了哈夫曼的树这一算法，并对我们生成哈夫曼的树进行编码，在这一过程中我遇到了许多问题：

在编写选择两个最小节点时，我忘记及时更新次小值，导致合并结果错误：

```

void Select(HuffmanTree HT, int n, int *s1, int *s2) {
    int min1 = 1000, min2 = 1000; // 初始值不够大
    *s1 = *s2 = 0;

    for (int i = 1; i <= n; i++) {
        if (HT[i].parent == 0) {
            if (HT[i].weight < min1) {
                // 错误点：未将原 min1 赋给 min2
                min1 = HT[i].weight;
                *s1 = i;
            } else if (HT[i].weight < min2) {

```

```
        min2 = HT[i].weight;
        *s2 = i;
    }
}
}
```

最后即时发现，解决了错误。

同样的，这里仍然存在一些可以改进的地方，在实际运用中，我们是通过哈夫曼编码来压缩数据，但当我们解码时，即从上到下遍历哈夫曼的树时，我们则必须依赖 lchild 和 rchild 逐层查找，无法直接回溯。为了解决这个，我们可以在连接子节点与父节点使用双向链表，这样会加大内存的开销但对解码很有帮助。